Lane Analysis & Wrong-Way Driving Specification

Technical Guide to Polynomial Lane Tracking, Ego Corridor Geometry, and Trajectory Conflicts

METRIC / SCOPE	SPECIFICATION / TARGET
Subsystem: Lane & Wrong-Side Safety	Module Path: core_safar_logic/road_analysis
Ego Corridor Half-Width: 1.05m	Total Corridor Width: 2.10m
Adjacent Lane Threshold: |X| > 1.85m	Test Pass Rate: 100% (6 / 6 Unit Tests)

1. Subsystem Architecture & Lane Corridor Geometry

The Lane and Trajectory Analysis Subsystem projects the predicted driving corridor of the ego vehicle and monitors surrounding lane boundaries. By establishing 4 lateral spatial zones, it distinguishes between oncoming traffic legally traveling in adjacent lanes and dangerous wrong-way drivers invading the ego path.

2. The 4 Lateral Driving Corridor Zones

Lateral offsets X relative to the vehicle centerline are segmented into deterministic zones:

- **Zone 1: Direct Ego Path ($|X| \le 1.05\text{m}$):** Any obstacle in this zone is on a direct collision course.
- **Zone 2: Near Margin ($1.05\text{m} < |X| \le 1.45\text{m}$):** High-risk buffer zone requiring warning.
- **Zone 3: Far Margin ($1.45\text{m} < |X| \le 1.85\text{m}$):** Cautionary awareness zone.
- **Zone 4: Road Shoulder / Adjacent Lane ($|X| > 1.85\text{m}$):** Safe. Zero false-alarm braking permitted.

3. Source Code Files Breakdown

FILE NAME	RESPONSIBILITY & ARCHITECTURAL WORK
road_analysis/wrong_side_detector.py	Implements `WrongSideDetector`. Analyzes target heading angle θ_{target} against ego heading θ_{ego}, calculates closing velocity v_{closing}, and determines if target is invading the ego lane.
road_analysis/lane_analyzer.py	Implements `LaneAnalyzer`. Uses 2nd-order polynomial curve fitting ($y = ax^2 + bx + c$) to estimate lane center, road curvature radius, and lateral vehicle offset.
python_perception/ego_path.py	Implements `EgoPathPredictor`. Projects vehicle width (2.10m total corridor width) along steering angle arc $\kappa = \frac{\tan(\delta)}{L}$ up to 50 meters forward.

4. Line-by-Line Code Walkthrough: Wrong-Side Detection

```
class WrongSideDetector:
    def detect_wrong_side(self, ego_state: VehicleState, target: Obstacle) -> Tuple[bool, float, str]:
        # 1. Check lateral corridor: if target is outside ego corridor (|X| > 1.85m), it is in its own lane
        if abs(target.position.x) > 1.85: return False, 99.0, 'Adjacent lane: safe'
        # 2. Heading alignment conflict: delta_yaw ~ 180 degrees indicates head-on oncoming traffic
        delta_yaw = abs(target.yaw_deg - ego_state.yaw_deg)
        is_opposing = (150.0 <= delta_yaw <= 210.0)
        if not is_opposing: return False, 99.0, 'Same direction or crossing traffic'
        # 3. Closing velocity: v_rel = v_ego + v_target
        closing_speed = ego_state.speed_mps + target.speed_mps
        ttc = target.position.z / max(0.1, closing_speed)
        return (ttc <= 3.0), ttc, f'CRITICAL: Wrong-side vehicle approaching at {closing_speed*3.6:.1f} km/h'
```

Code Explanation: Line 4 immediately filters out traffic in opposing lanes ($|X| > 1.85\text{m}$). Lines 6-8 confirm opposing heading vectors ($150^\circ - 210^\circ$). Lines 10-12 compute extreme closing speed ($v_{\text{ego}} + v_{\text{target}}$) and trigger emergency intervention when $\text{TTC} \le 3.0\text{s}$.